

SDLC Using **Agentic AI**

Building an Adaptive AI Software Engineering Organization

Presented By:

Abhishek Raj (M.Tech Intern)

Guide:

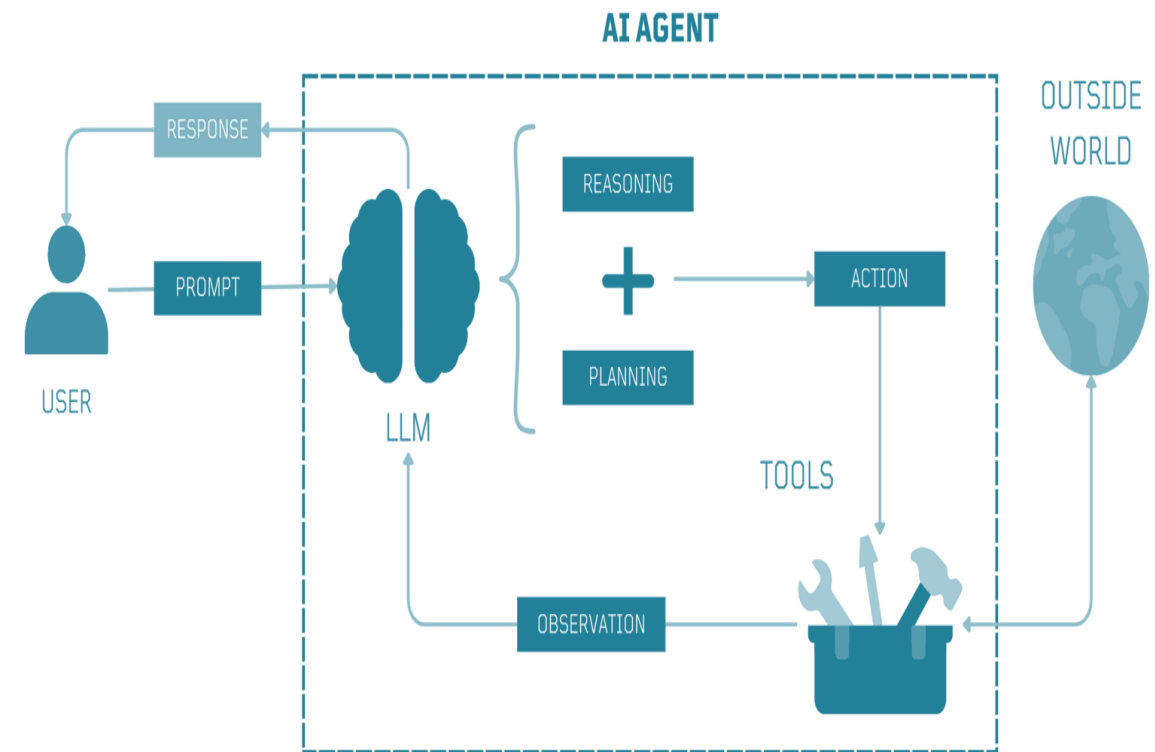
M.B. Sai Charan (Scientist C)

What is an AI Agent?

An AI agent is a system where an LLM operates in a loop: it receives observations from an environment (user messages, tool outputs, sensor data), reasons about what to do next, takes actions (tool calls, code execution, API requests), and iterates until a goal is achieved or it explicitly asks for human input.

An AI agent should have the following characteristics:

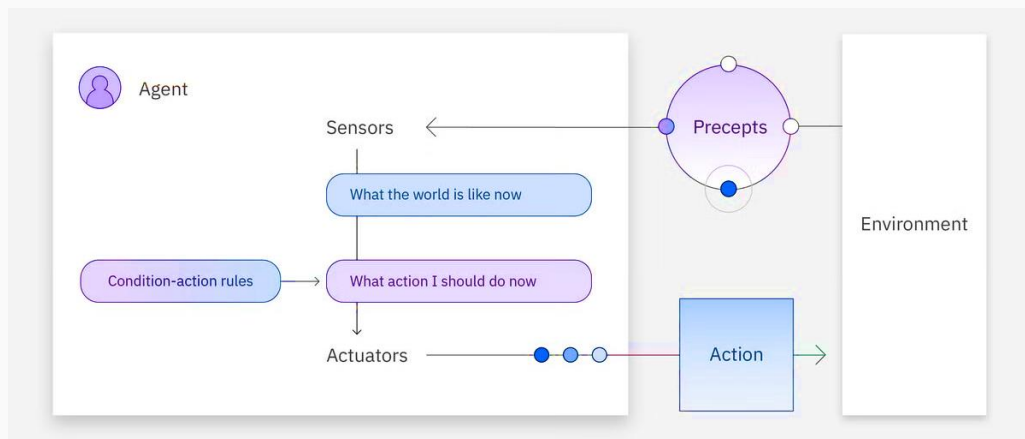
- **Persistence:** An agent must remember what it has done, what failed, and what context was established—across turns, sessions, and even days.
- **Grounding:** The agent must access up-to-date, domain-specific knowledge that was not present in its training data.
- **Action:** The agent must interact with external systems—databases, APIs, file systems, browsers—through well-defined interfaces.
- **Coordination:** Complex tasks often exceed what a single agent can handle; multiple specialized agents must collaborate, delegate, and negotiate.
- **Safety:** Autonomous action requires guardrails, human oversight, and graceful degradation when the agent is uncertain. [6]



Types of AI Agents

Simple Reflex Agents

Simple reflex agents are the simplest agent form that grounds actions on [perception](#). This agent does not hold any memory, nor does it interact with other agents if it is missing information. [4]



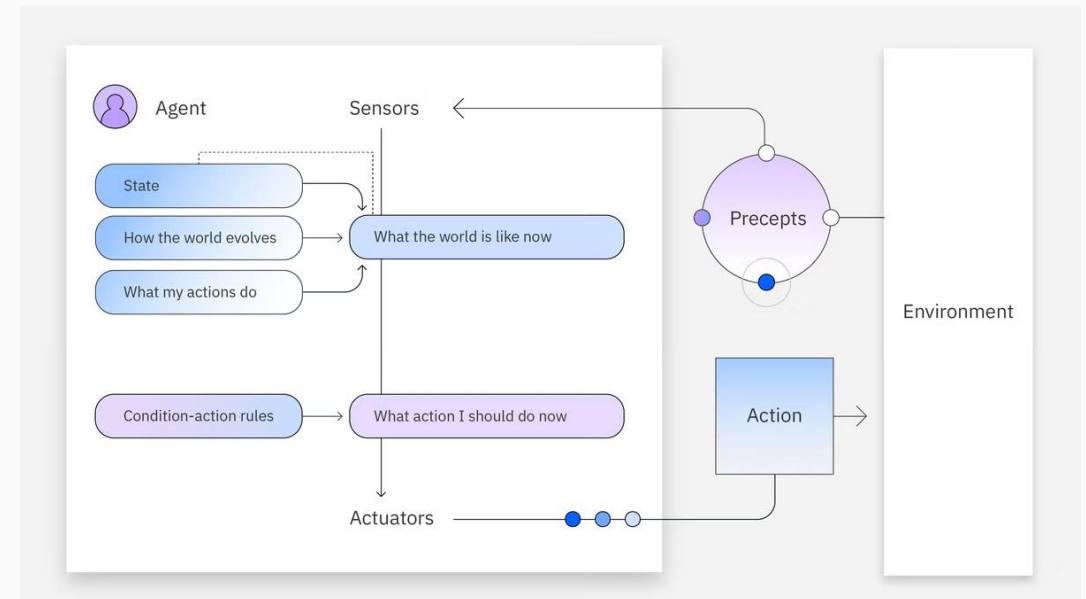
- ✓ No memory capacity
- ✓ Rule-based decisions
- ✓ Fast but inflexible

EXAMPLES

Auto doors, Thermostats, Motion lights

Model-Based Agents

Model-based reflex agents use both their current perception and memory to maintain an internal model of the world. As the agent continues to receive new information, the model is updated. The agent's actions depend on its model, reflexes, previous precepts and current state. [4]



- ✓ Maintains internal state
- ✓ Handles partial observability
- ✓ Updates knowledge continuously

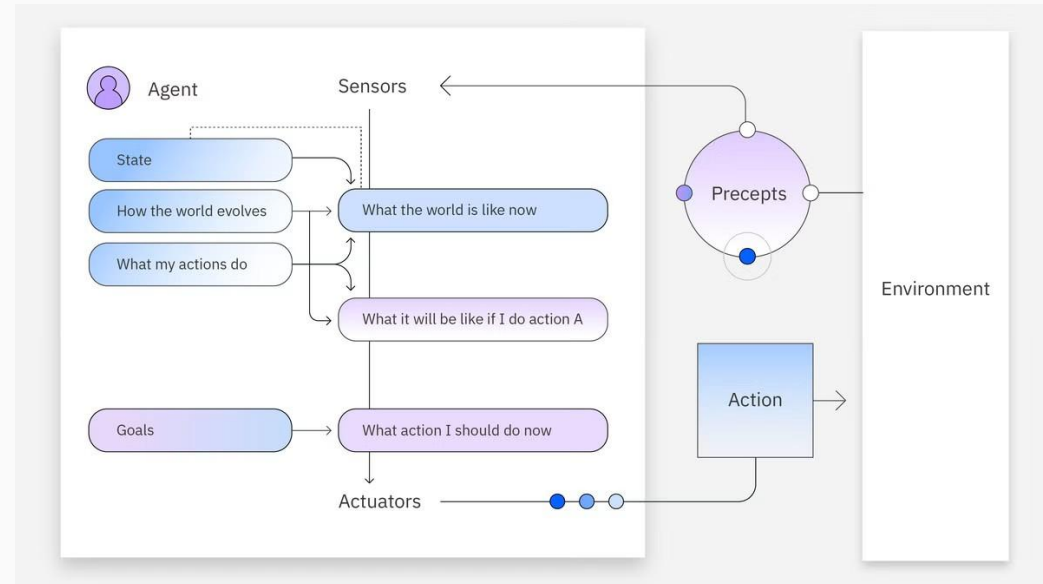
EXAMPLES

Robot vacuums, Warehouse robots

Types of AI Agents (Cont.)

Goal-Based Agents

Goal-based agents have an internal model of the world and also a goal or set of goals. These agents search for action sequences that reach their goal and plan these actions before acting on them. This search and planning improve their effectiveness when compared to simple and model-based reflex agents. [4]



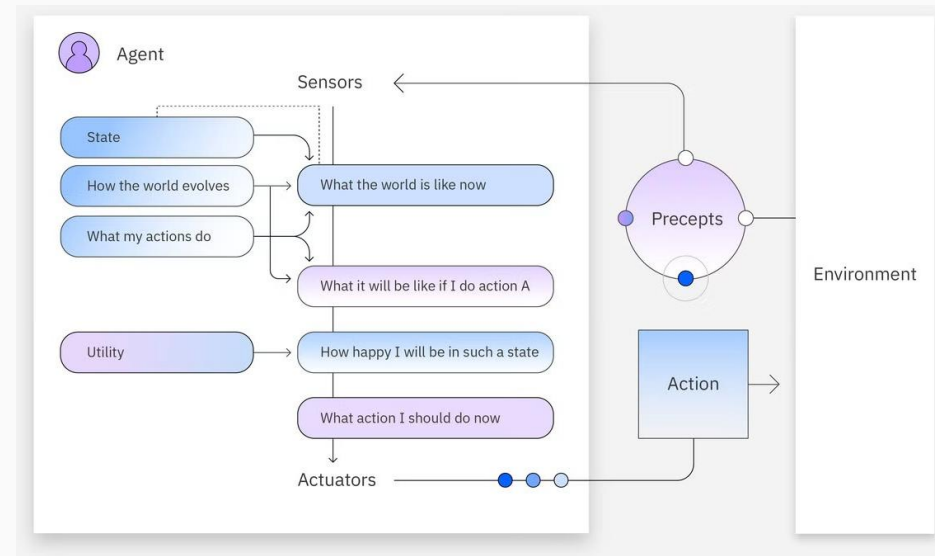
- ✓ Goal-oriented approach
- ✓ Planning & search capabilities
- ✓ Evaluates possible actions

EXAMPLES

GPS navigation, Logistics planners

Utility-Based Agents

Utility-Based Agents evaluate multiple possible actions using a utility function, selecting the option that maximizes overall performance and handles trade-offs efficiently. [4]



- ✓ Decision optimization
- ✓ Handles complex trade-offs
- ✓ Maximizes expected reward/utility

EXAMPLES

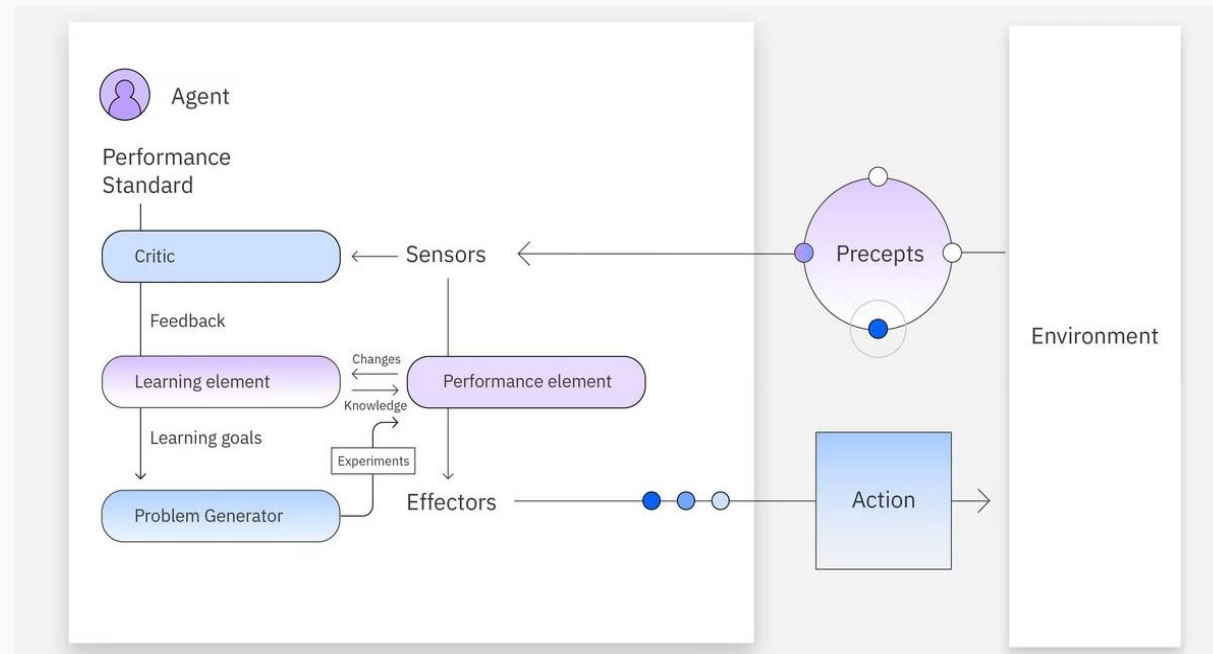
Navigation Systems

Types of AI Agents (Cont.)

Learning Agents

Learning agents hold the same capabilities as the other agent types but are unique in their ability to learn. New experiences are added to their initial knowledge base, which occurs autonomously. This learning enhances the agent's ability to operate in unfamiliar environments. Learning agents might be utility or goal-based in their reasoning and are composed of four main elements:

- **Learning:** This process improves the agent's knowledge by learning from the environment through its precepts and sensors.
- **Critic:** This component provides feedback to the agent on whether the quality of its responses meets the performance standard.
- **Performance:** This element is responsible for selecting actions upon learning.
- **Problem generator:** This module creates various proposals for actions to be taken. [4]



- ✓ Self-improving over time
- ✓ Learns directly from experience
- ✓ Adapts to changing environments

EXAMPLES

AlphaGo, Netflix Recommendations, Adaptive spam filters

Architectures

Workflow vs Autonomous

WORKFLOW PATTERNS



Deterministic Control

Execution paths, step ordering, and condition gates are explicitly defined in code. LLM calls serve as reliable processing nodes within the system pipeline. [6]

- Higher predictability and testability
- Lower latency and controlled token expenditure
- Ideal for standardized, step-by-step transformations

Primary Patterns: Prompt Chaining, Sequential Workflows, Routing, Parallelization, Orchestrator-Workers, Evaluator-Optimizer.

AUTONOMOUS PATTERNS



Dynamic Reasoning Loops

The agent autonomously determines its execution path based on environment feedback, goal planning, reflection, and tool execution. [6]

- High adaptability to open-ended goals
- Dynamic tool selection and runtime recovery
- Handles complex multi-step reasoning environments

Primary Patterns: ReAct (Reason+Act), Planning Agents, Reflection / Self-Critique, Multi-Agent Collaboration.

Workflow Patterns

1. Prompt Chaining

Decomposes a task into a fixed sequence of LLM calls where the output of one step becomes the input context for the next, with validation gates, ensuring higher precision and intermediate quality control at each stage. [6]

Input → Analyze → Transform → Validate → Generate

Use Cases: Multi-stage reasoning, document analysis, data transformation.

2. Sequential Workflow

Arranges steps into a strict, deterministic assembly line where structured data outputs pass directly between modular code stages, maintaining strict execution order and data schema consistency across stages. [6]

Extract → Transform → Analyze → Generate

Use Cases: Document processing pipelines, automated report generation

3. Routing

Uses an initial router or classifier to direct incoming requests to specialized agents or processing paths with tailored prompts and tools, optimizing latency and cost by serving simple requests quickly and reserving larger, specialized models for complex tasks. [6]

Request → Router → Specialist → Response

Use Cases: Customer support triage, query classification

4. Parallelization

Distributes independent subtasks across concurrent LLM calls. Executes simultaneously and aggregates outputs to reduce total system latency, making it ideal for independent evaluation tasks, majority-voting consensus mechanisms, and multi-perspective verification. [6]

Input → Split →

→ Security
→ Performance
→ Quality

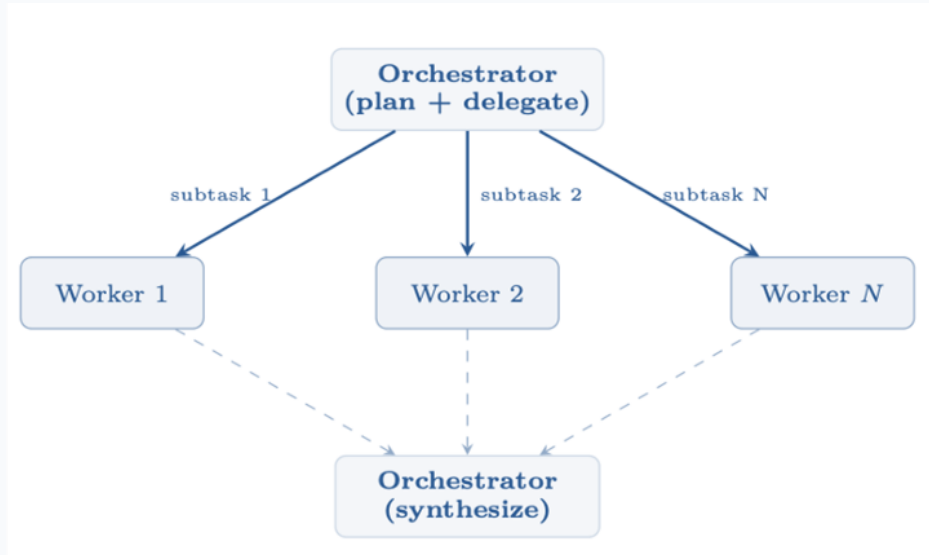
 → Aggregate → Output

Use Cases: Multi-aspect code reviews, guardrail checks.

Workflow Patterns (Cont.)

5. Orchestrator–Workers

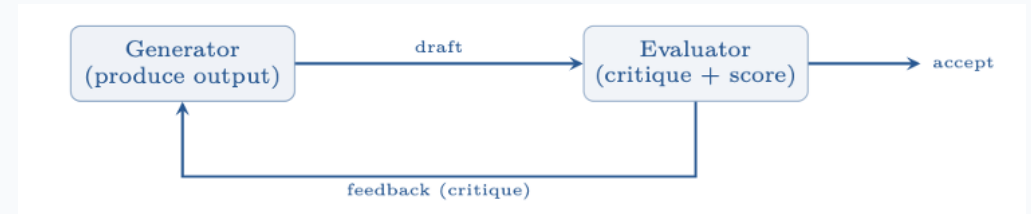
A central LLM dynamically analyzes a task, breaks it down into subtasks, delegates to specialized worker agents, and synthesizes the collected worker outputs into a final result. [6]



Example Use Cases: Complex software development, automated research synthesis, multi-domain compliance analysis.

6. Evaluator–Optimizer

Creates an iterative loop where a Generator LLM produces output and an Evaluator LLM assesses it against quality criteria, providing actionable feedback until thresholds are met.[6]



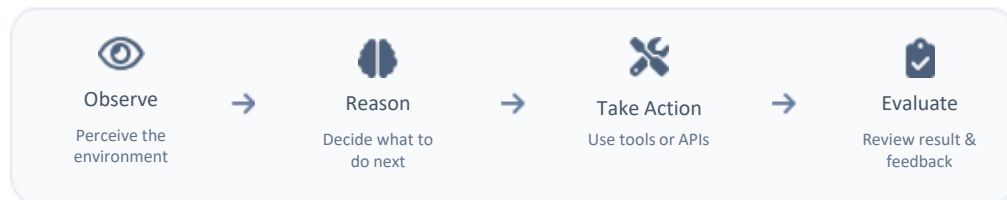
Example Use Cases: Code generation with test pass verification, high-stakes translation, quality-controlled drafting.

Autonomous Patterns

Systems where the model independently determines execution paths, tool selection, reflection loops, and agent collaboration.

1. ReAct (Reason & Act)

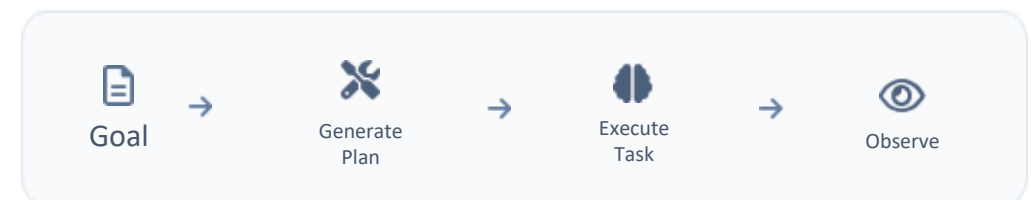
It combines reasoning and action in a dynamic, iterative loop where a single agent observes the environment, formulates a plan, executes an action using available tools or APIs, and evaluates the resulting feedback to decide the next step. By constantly re-evaluating its context, the model can course-correct on the fly, retry failed tool calls, or adjust its strategy based on intermediate outputs. [6]



Example Use Cases: Q&A, Code generation & debugging

2. Planning Agent

It generates an explicit plan for achieving a goal before beginning execution. The plan contains tasks and dependencies, while the agent can revise the plan when new information, failures, or unexpected results are encountered during execution.[6]



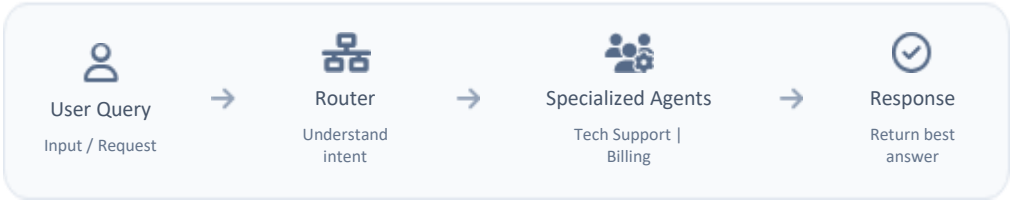
Example Use Cases: Content generation, document analysis, data transformation

Autonomous Patterns (Cont.)

Systems where the model independently determines execution paths, tool selection, reflection loops, and agent collaboration.

3. Reflection & Self-Critique

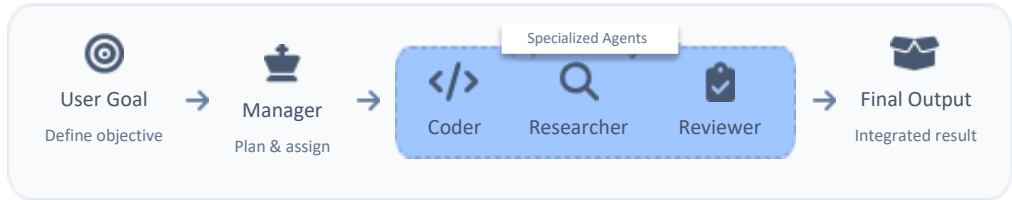
It introduces a self-evaluation stage where the agent examines its previous actions or output, identifies errors or inefficiencies, and modifies its strategy. The reflection can also be stored as memory and reused during subsequent attempts [6]



Example Use Cases: Customer support automation

4. Multi-Agent Swarm

A **Multi-Agent Swarm** relies on a manager agent to orchestrate a ecosystem of domain-expert sub-agents working together toward a high-level goal. The manager breaks down complex user objectives into specific tasks and assigns them to specialized roles—such as a **Coder** for implementation, a **Researcher** for context gathering, and a **Reviewer** for quality assurance—who can debate, review each other's work, and iteratively refine deliverables. [6]



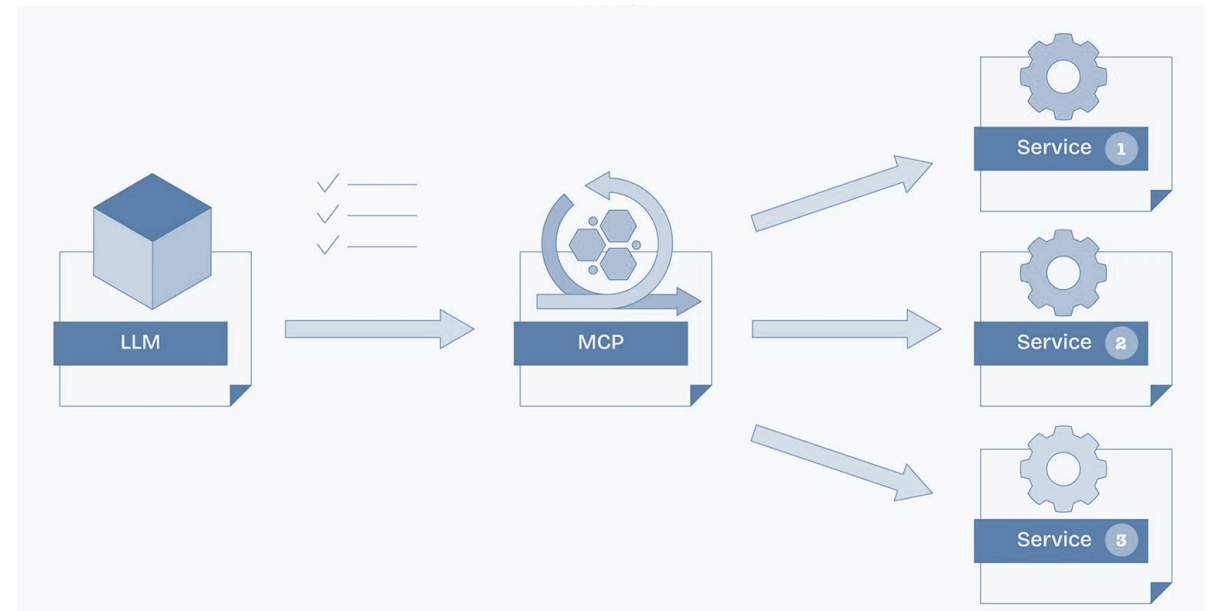
Example Use Cases: Software development teams, Threat analysis & support

What is MCP & Why Do We Need It?

- **The Core Problem:** LLM knowledge is frozen at training time & LLMs Agents cannot inherently interact with the outside world (real-time data, business systems, APIs).
- **The Solution (MCP):** The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers [18]

Here is a simplified look at how MCP would handle this:

1. **Discover Tools:** AI realizes it needs external help and finds the database and email tools registered on MCP.
2. **Fetch Data:** AI requests the sales report via MCP; the database server fetches and returns the report.
3. **Send Email:** AI hands the report to the email tool, which sends it to the manager.
4. **Confirm:** AI tells you the task is finished. [18][6]

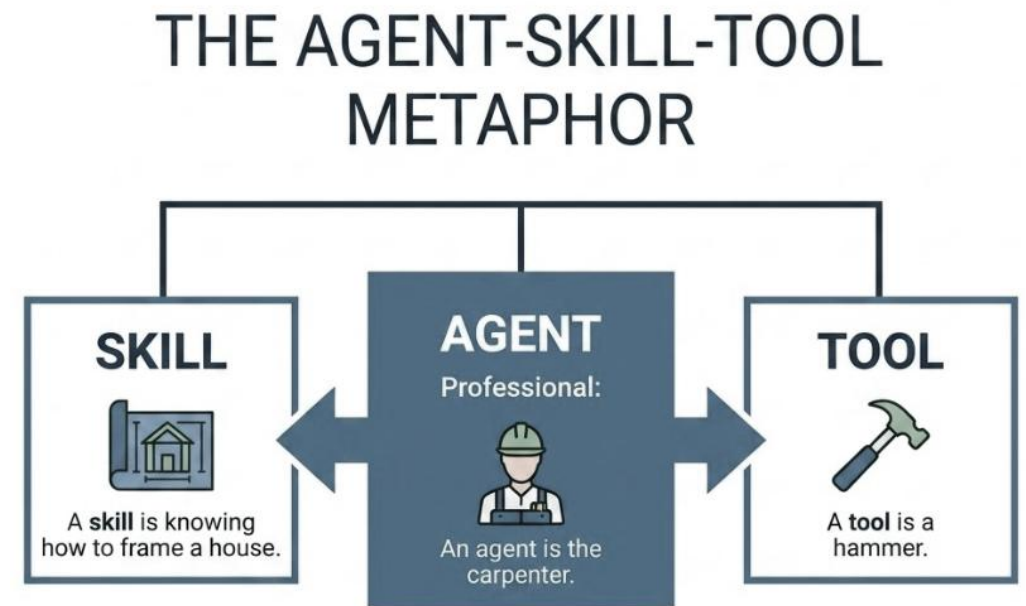


Agent Skills: Composable Capabilities

What Is a Skill?

A skill is a self-contained capability module that gives an agent expertise in a specific domain or task. Unlike a raw tool (which exposes a single function), a skill encompasses:

- **System prompt augmentation:** Domain-specific instructions, constraints, and persona elements injected into the agent's context.
- **Tool bindings:** One or more tools the skill requires (APIs, MCP servers, local commands).
- **Knowledge:** Reference material, examples, or few-shot demonstrations the agent needs to execute the skill correctly.
- **Workflow logic:** Multi-step procedures, decision trees, or conditional flows that guide the agent through complex tasks.
- **Guardrails:** Skill-specific safety constraints, output format requirements, and validation rules. [6]



Agentic Frameworks

Framework	Execution Model	Core Strengths	Key Limitations	Ideal Use Cases	Offline / Local Capability
LangChain [7]	Chained Prompting & Tools	Rich toolset, extensive RAG ecosystem, vast API integrations	Workflow complexity at scale, abstractions can hide errors	General LLM applications, retrieval-augmented generation	Excellent: Natively connects to local LLMs (Ollama, Llama.cpp) and local vector stores (Chroma).
LangGraph [8]	Stateful Graph Orchestration	Cycles and loops, robust state persistence, time-travel debugging	Steeper learning curve, excessive setup for simple tasks	Production-grade multi-agent systems, human-in-the-loop control	Excellent: Inherits LangChain's local capabilities; operates entirely offline with self-hosted models.
CrewAI [9]	Role-based Team Collaboration	Easy setup, intuitive agent personas, automatic task delegation	Limited dynamic branching and complex workflow flow control	Autonomous research pipelines, content creation, structured teams	Strong: Explicitly supports local execution by binding agents to local endpoints via Ollama or LiteLLM.
OpenAI Agents SDK [10]	Lightweight Handoffs & Guardrails	Minimal abstractions, built-in tracing, standardized routing primitives	Lacks built-in persistent long-term memory or native graph structures	Rapid deployment of production agents, voice pipelines, clear delegation	Very Good: Provider-agnostic design allows it to run offline by routing to local models via the Chat Completions API.

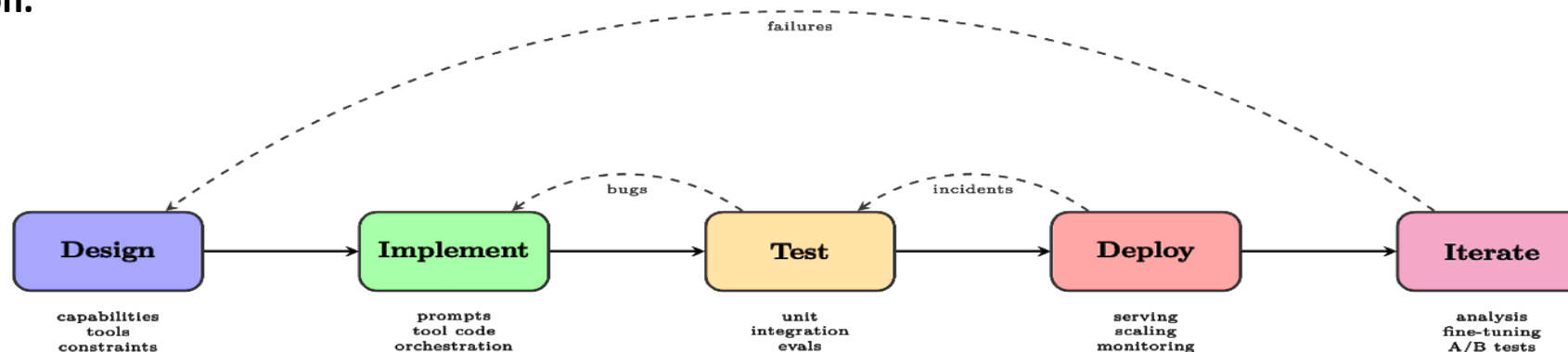
The Traditional SDLC

The Traditional Software Development Life Cycle (SDLC) is the structured framework that engineering teams follow to plan, design, build, test, and deploy software applications from initial concept.



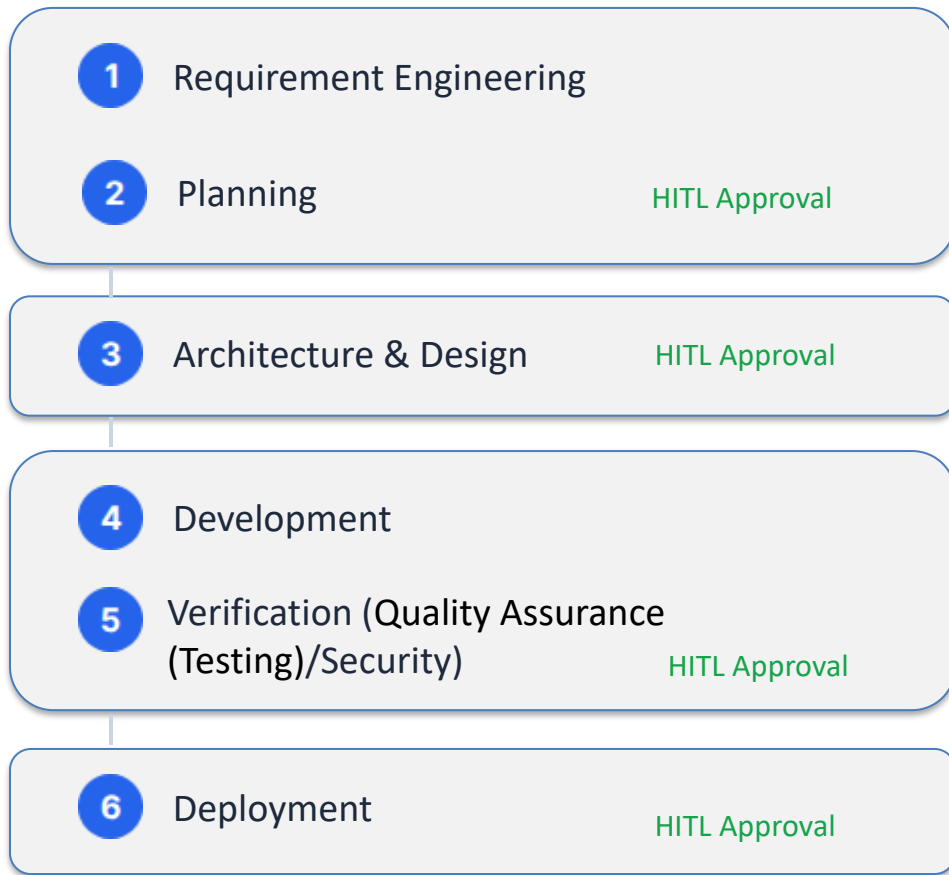
This Manual SDLC execution creates severe friction—engineers waste most of their time on administrative handoffs, environment setups, regression testing, and ticketing overhead instead of writing core logic.[5]

So, Our Aim is to automate the Traditional SDLC with Agentic AI, which will replace these static bottlenecks with continuous, self-healing pipelines that autonomously execute tests, patch build failures, flag vulnerabilities, and manage releases. This **eliminates human error, compresses cycle times from months to hours, and frees development teams to focus on high-value architecture and innovation.**



Proposed Architecture: Modular Pipeline

A modular pipeline with integrated **Human-in-the-Loop (HITL)** governance gates to ensure enterprise safety.

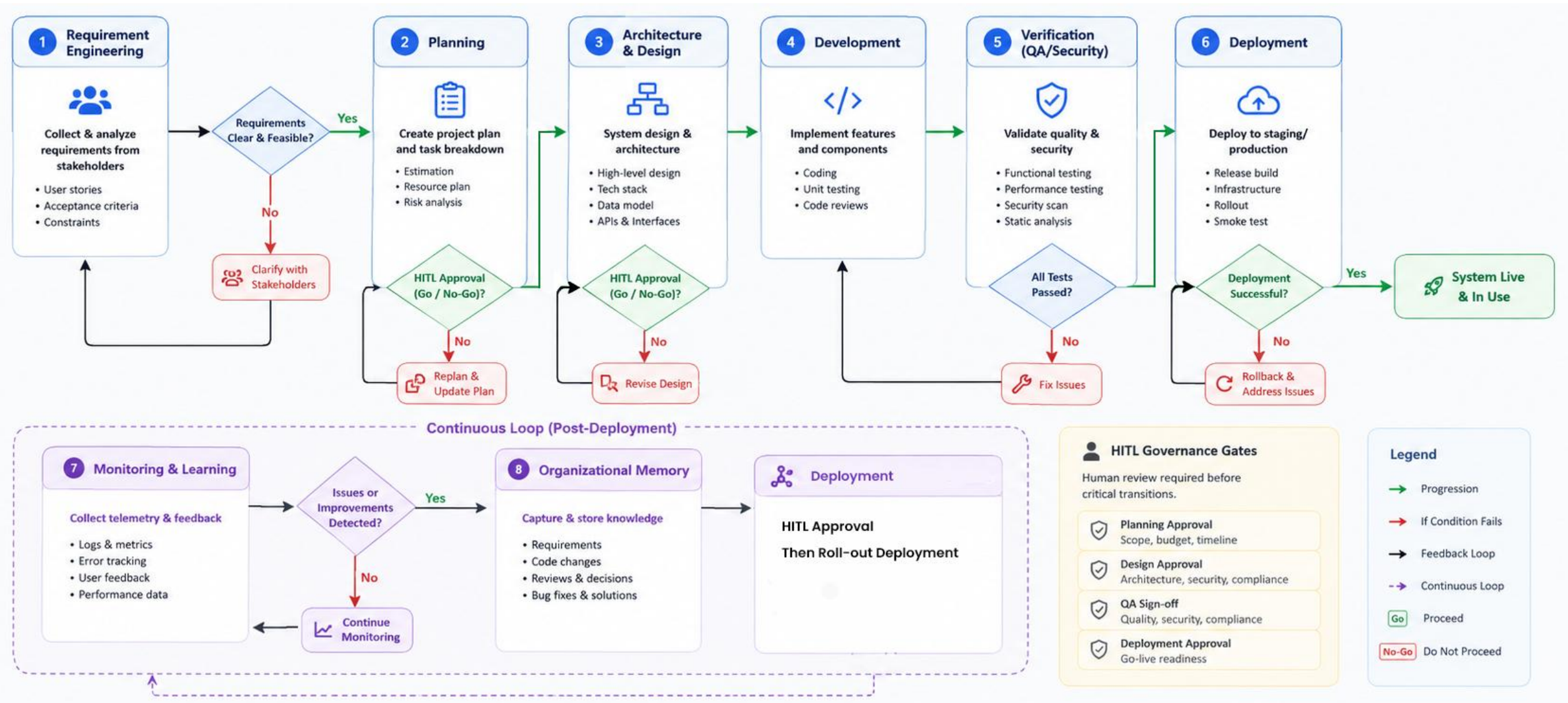


Continuous Loop

7. Monitoring & Learning: Consumes production logs, metrics, and user feedback to detect errors and optimize prompts.

8. Organizational Memory: Captures all requirements, code, reviews, and bug fixes into a Knowledge Graph for cross-sprint learning.

Proposed Architecture: Modular Pipeline



Identifying the Gap

While highly capable, existing autonomous systems index heavily on code generation, lacking end-to-end SDLC orchestration and organizational memory.

System	Requirements	Design	Coding	Testing	Deployment
Devin / OpenHands [14][15]	Partial	Partial	Yes	Yes	Partial
MetaGPT / ChatDev [11][12]	Yes	Yes	Yes	Partial	No
SWE-Agent [13]	No	No	Yes	Yes	No
Proposed System	Yes	Yes	Yes	Yes	Yes

Core Innovation: Adaptive Team Generation

Traditional: Fixed Agents

Standard agentic systems rely on rigid, hardcoded workflows (e.g., always Planner → Developer → Tester).

- Inefficient for small tasks (over-engineered).
- Lacks specialized roles (Security, DevOps) for complex enterprise apps. [1]

Proposed: Dynamic Generation

The Orchestrator analyzes project complexity, risk, and size before execution to spawn agents on-the-fly.

- **Small Script:** Spawns Planner + Developer.
- **Enterprise App:** Spawns Architect, Backend, Frontend, Security, QA, DevOps.
- Agents are retired when tasks complete, saving compute.

Core Innovation: Hierarchical Software Knowledge Graph (HSKG)

Beyond Traditional Memory:
A Context-Aware Project Memory

The Problem: Current AI agents retrieve entire repositories or large documents, resulting in high token consumption and irrelevant context. [1]

Our Solution: Our proposed HSKG stores the software project as a hierarchical graph, enabling agents to retrieve only the smallest relevant subgraph required for a task.



Project Graph

Project

- └ Authentication
 - Login, Register
- └ Inventory
 - Product, Stock
- └ Invoice
 - Create, Payment

Purpose

- Represents the complete SDLC structure
- Maintains hierarchy between project components
- Enables modular navigation



Context-Aware

Instead of:
Entire Repository → LLM

Agent retrieves:
Login Module → UI → Backend Script & Variables
→ API → User DB

Benefits

- Minimal context
- Lower token usage
- Faster reasoning
- Better accuracy
- Reduced hallucination

Local Offline Execution Strategy

Deploying via offline models ensures data privacy. Model selection is based on VRAM footprint, Training data freshness, Suitable for Agentic Operations.

Model (4-bit Quant)	Recommended Hardware (VRAM)	Knowledge Cutoff	Agent Capability	Best Use Case
Gemma 4 4B	Laptop / iGPU (~5 GB)	Early 2026	★ ★ ★	Basic assistants, RAG, lightweight offline agents
Gemma 4 12B	RTX 4070 (8–10 GB)	Mid 2026	★ ★ ★ ★	General-purpose AI agents with good reasoning & tool use
Mistral Small 3.2 (24B)	RTX 4090 (16 GB)	Mid 2025	★ ★ ★ ★ ★	Enterprise agents, multi-tool workflows, planning
Qwen3-Coder 32B	Workstation / Mac M-Series (20–22 GB)	Mid 2025	★ ★ ★ ★ ★	Coding agents, debugging, software engineering
Llama 3.3 70B	Multi-GPU / High-End Server (48+ GB)	Late 2023*	★ ★ ★ ★ ★	Research, complex planning, multi-agent orchestration

Source: Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.

Conclusion

We are proposing an **Autonomous Software Development Lifecycle (A-SDLC)** framework that transforms traditional software engineering into an **AI-Orchestrated Software Engineering Organization**.

Instead of isolated coding assistants, the system coordinates multiple specialized AI agents across the complete SDLC—from **requirements engineering** to **continuous monitoring and learning**.

Expected Outcomes

- ✓ Reduced context size and token consumption
- ✓ Faster and more accurate software development
- ✓ Improved traceability across the SDLC
- ✓ Persistent engineering knowledge

References

Research Papers & Book

- [1] He, J., Treude, C., & Lo, D. (2025). *LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision...* ACM TOSEM.
- [2] Apostolou, S. A., et al. (2026). *Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC*. arXiv.
- [3] *Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence...* (2026). arXiv.
- [4] Veselka Sasheva Petrova-Dimitrova, "Classifications of intelligence agents and their applications," *Fundamental sciences and applications*, Vol. 28, No. 1, 2022.
- [5] Sommerville, I. *Software Engineering (10th Edition)*.
- [6] The Hitchhiker's Guide to Agentic AI: From Foundations to Systems by [Haggai Roitman](#)

Articles & Official Documentations

- [7] *LangChain Documentation*.
- [8] *LangGraph Documentation*.
- [9] *CrewAI Documentation*.
- [10] *OpenAI Agents SDK Documentation*.
- [11] Hong, S., et al. (2024). *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*.
- [12] Qian, C., et al. (2024). *ChatDev: Communicative Agents for Software Development*.
- [13] Yang, J., et al. (2024). *SWE-Agent: Agent-Computer Interfaces Enable Automated SE*.
- [14] *OpenHands Project*.
- [15] Cognition AI – *Devin: AI Software Engineer*.
- [16] *Microsoft-copilot/for-individuals/do-more-with-ai/general-ai/understanding-ai-agents-vs-chatbots?*
- [17] IBM: <https://www.ibm.com/think/topics/ai-agents>
- [18] Google Cloud: cloud.google.com/discover/what-is-model-context-protocol

Thank You